"Suspect Pattern Detector"

A Project Report Submitted in the partial fulfillment of the requirements of the course titled

"Problem Solving Through Programming (JAVA)"

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF FRESHMAN ENGINEERING

By

FATHIMA QHIBTIYA KHADER	**2520030155**
KUVIRA AMBALA	**2520030192**
KASULA YASHAS	**2520030401**
GOTTIPATI EESHA YUKTHA	**2520030436**
KARUTURI MAHAISWARYA	**2520030550**
DEVAJI RASHMIKA	**2520090152**

Under the Esteemed Guidance of

Guide Name: Mamatha Gadde
Designation
Department of Freshman Engineering

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF FRESHMAN ENGINEERING

Declaration

The Project Report entitled **" Suspect Pattern Detector "** is a record of Bonafide work of **Fathima Qhibtiya Khader– 2520030155, Kuvira Ambala – 2520030192, KASULA YASHAS – 2520030401, Gottipati Eesha Yuktha – 2520030436, Karuturi Mahaiswarya– 25200030550, Devaji Rashmika – 2520090152** submitted in partial fulfillment of the requirements of the course titled "Problem Solving Through Programming (JAVA)" under the B.Tech Ist Year Trimester - I program in Department of Freshman Engineering at K L University. The results presented in this report have not been copied from any other department, university, or institute.

FATHIMA QHIBTIYA KHADER – 2520030155

KUVIRA AMBALA – 2520030192

YASHAS KASULA – 2520030401

GOTTIPATI EESHA YUKTHA – 2520030436

MAHAISWARYA KARUTURI - 2520030550

DEVAJI RAHMIKA – 2520090152

K L (Deemed to be) University

DEPARTMENT OF FRESHMAN ENGINEERING

CERTIFICATE

This is certify that the java project based report entitled **"Suspect Pattern Detector"** is a bonafide work done and submitted by **Fathima Qhibtiya Khader – 2520030155, Kuvira Ambala – 2520030192, Kasula Yashas– 2520030401, Gottipati Eesha Yuktha– 2520030436, Karuturi Mahaiswarya– 2520030550, Devaji Rashmika– 2520090152** in partial fulfillment of the requirements of the course titled "Problem Solving Through Programming (JAVA)" under the B.Tech Ist Year Trimester - I program in Department of Freshman Engineering, K L (Deemed to be University), during the academic year **2025-2026.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who have assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Y Ashok**, and my guide Mrs. Mamatha Gadde**,** Department of Freshman Engineering which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. N. Chaitanya Kumar**, Head of the ***Department for Freshman Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

FATHIMA QHIBTIYA KHADER – 2520030155

KUVIRA AMBALA – 2520030192

YASHAS KASULA – 2520030401

GOTTIPATI EESHA YUKTHA – 2520030436

MAHAISWARYA KARUTURI - 2520030550

DEVAJI RAHMIKA – 2520090152

SUSPECT PATTERN DETECTOR

ABSTRACT

Crime is a pervasive societal issue that necessitates effective and efficient management by law enforcement. Traditional manual record-keeping systems are often time-consuming, prone to error, and lack advanced analytical capabilities, hindering the speed of investigations and the ability to detect emerging crime trends.

Suspect Pattern Detection is a Java-based application that scans textual data for specific patterns associated with suspicious or sensitive content. The system processes input text line by line and identifies occurrences of predefined keywords or phrases. Each match is recorded along with its location in the text, and a summary report is generated to document the findings. The application operates by comparing input data against a list of suspect patterns and flags any matches found during the scan. It is structured to handle plain text files and produce a clear output indicating the presence and frequency of detected patterns. The project focuses on basic text analysis and pattern recognition within a controlled dataset.

This project offers a foundational approach to automated pattern detection in textual records, providing a streamlined method for identifying potentially critical information. By reducing manual effort and enhancing consistency in data review, Suspect Pattern Detection contributes to the broader goal of improving investigative efficiency and supporting early identification of crime-related indicators.

Index

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Crime continues to be a major challenge for society and managing it effectively requires timely detection and analysis. As online communication continues to grow, people frequently exchange messages through emails, chats, and social media. While this makes communication faster and more convenient, it also increases the chances of encountering scams, fraud attempts, and misleading messages. Many users struggle to identify suspicious content, especially when scammers use subtle or deceptive language. This project introduces a simple, yet practical Java application designed to support this effort by identifying suspicious or potentially harmful content within text data.

The system works by scanning sources such as chat logs, emails, or written documents and searching for keywords or patterns commonly linked to fraud, scams, or malicious intent. By using basic pattern-matching techniques, the application helps users quickly spot content that may require further attention.

To address this challenge, this project focuses on building a simple Java-based system that automatically scans text and evaluates the risk of a potential scam. Instead of relying on complex machine learning models, the system uses basic pattern-matching techniques to detect keywords, phrases, or writing styles commonly associated with fraudulent behavior.

By analyzing the text and assigning a risk level -- such as **low**, **medium**, or **high.** The system helps users quickly understand whether a message might be unsafe. This makes it easier for individuals to stay alert, avoid scams, and make informed decisions when interacting with unfamiliar or suspicious content.

The project aims to provide a lightweight, easy-to-use tool that demonstrates how simple text analysis can support safer digital communication.

1.2 Problem Statement

Users often struggle to manually identify suspicious content in large volumes of messages,

documents, or logs. At the same time, law enforcement and analysts need better ways to understand crime trends and predict potential risks using historical data. Without accessible tools, detecting harmful patterns becomes slow, inefficient, and prone to human error.

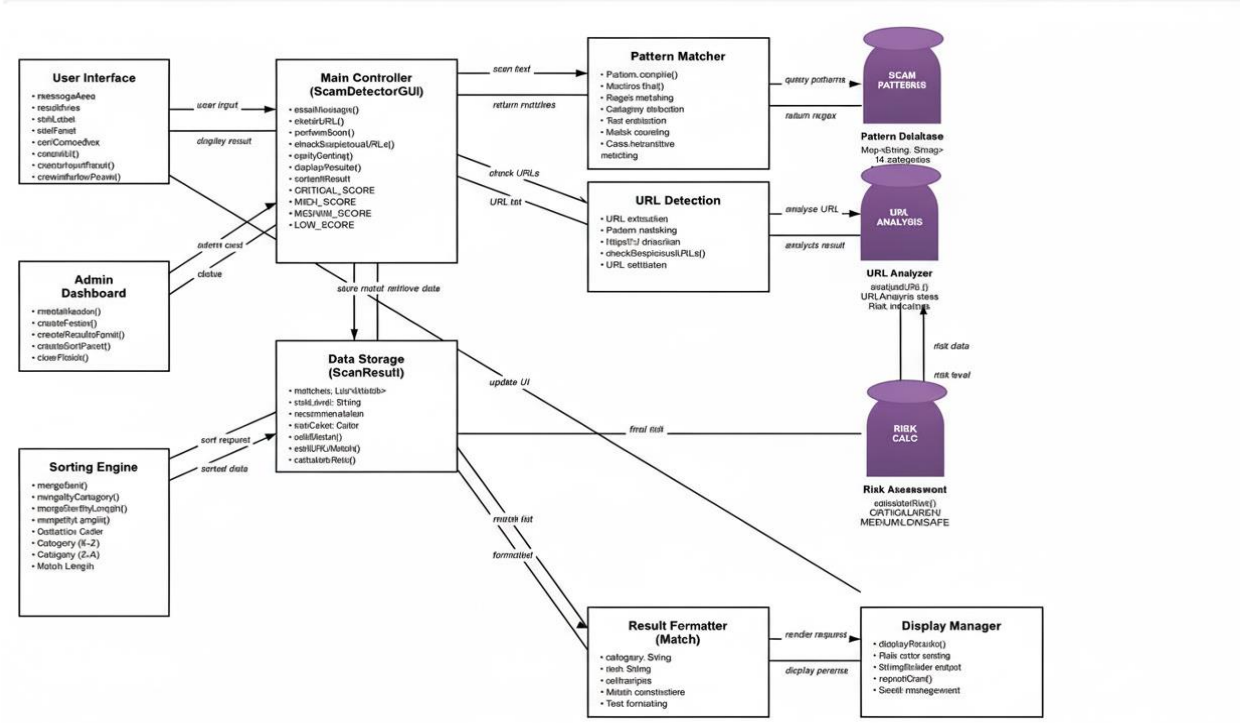
With the rise of digital communication, people are constantly exposed to messages from unknown sources through emails, social media, and online platforms. While many of these messages are harmless, a significant number is designed to deceive users through scams, phishing attempts, and fraudulent schemes. These scam messages often appear convincing, making it difficult for individuals, especially non-technical users to identify potential risks.

Manually checking every message for suspicious content is not practical, especially when dealing with large volumes of text. Users may overlook subtle warning signs, such as specific keywords, unusual phrasing, or manipulative language commonly used by scammers. At the same time, law enforcement and analysts need better ways to understand crime trends and predict potential risks using historical data. Without accessible tools, detecting harmful patterns becomes slow, inefficient, and prone to human error.

This lack of awareness increases the chances of falling victim to fraud, financial loss, or data theft.

CHAPTER -2 SYSTEM ARCHITECTURE

2.1 High-level architecture diagram



2.2 Class Diagram


JFrame
<<Java Swing Superclass>>
extends
ScamDetectorGUI
+SCAM_PATTERNS
+messageArea
+resultArea
+riskLabel
+riskPanel
+sortComboBox
+currentResult
+scanMessage()
+checkURL()
+performScan()
+analyzeURL()
+mergeSort()
+mergeSortByLength()
+displayResults()
+clearFields()
contains
contains
uses
1
1
1
1
1
1
ScanResult
+matches
+riskLevel
+recommendation
+riskColor
+addMatch()
+addURLMatch()
+calculateRisk()
has
*
1
Match
+category
+text
+urlAnalysis
+Match(String, String)
+Match(String, String,
URLAnalysis)
has
*
0..1
URLAnalysis
+url
+isSuspicious
+riskLevel
+reasons
+URLAnalysis(String)

CHAPTER -3 CO's ATTAINMENT

3.1 CO1 Attainment

CO1 Syllabus	CO1 Concepts Included in Project
• Problem-solving methodology and flowchart design • Introduction to Java programming model (syntax, variables, data types, type casting, operators) • Input/output operations (Scanner, BufferedReader) • Control flow: if-else, nested if, switch-case • Iterative constructs: for, while, do-while • Logical reasoning and flow tracing through dry runs • Pattern printing and arithmetic-based problem logic • Debugging and error tracing techniques	Introduction to Java programming model (syntax, variables, data types, type casting, operators) Input/output operations (Scanner, BufferedReader) Control flow: if else ladder ;switch Iterative contructors : for loop;while loop

3.2 CO2 Attainment

CO2 Syllabus	CO2 concepts included in project
Concept of arrays and memory representation 1D array operations: creation, traversal, insertion, deletion, rotation, merging 2D arrays: matrix representation and manipulation Searching techniques -Linear & Binary search with complexity analysis Sorting techniques - Bubble, Selection,	Merge sort

Insertion, Merge, Quick Sort Optimization strategies: two-pointer technique, prefix sum, sliding window. Matrix algorithms -transpose, rotation, diagonal operations CodeChef-style problem solving using arrays and loops	

3.3 CO3 Attainment

CO3 Syllabus	**CO3 concepts included in project**
String handling and immutability. String vs StringBuilder vs StringBuffer Common string problems: palindrome, anagram, substring, frequency count, character manipulation. Regular Expressions (regex) -pattern matching and text validation Bitwise operators (AND, OR, XOR, NOT, shifts, masks). Bit manipulation tricks for optimization (checking even/odd, power of 2, swapping, subset generation). Recursion fundamentals, base & recursive cases, tracing stack frames Recursive problem-solving: factorial, Fibonacci, backtracking (n-Queens, subset sum) Quantitative and mathematics-based logical problems	String methods, Recursion, String Builder, Pattern Matching, Text validation

3.4 CO4 Attainments

CO4 Syllabus	CO4 concepts included in project
Transition from procedural to object-oriented logic. Defining and using classes & objects. Methods, parameters, return types. Method overloading, constructors, and use of this keyword. Access specifiers and encapsulation Static data and methods. Design of simple OOP-based applications (banking system, student result system, etc.). Modularization of logic through classes	Constructors, return type, methods, parameters, Access specifiers and encapsulation Static data and methods

3.5 CO5 Attainments

CO5 Syllabus	CO5 concepts included in project
Concept of inheritance and types. Method overriding, super, and final keyword. Abstract classes and abstract methods Interfaces and multiple inheritance in Java Polymorphism (compile-time and runtime). Dynamic binding and late method resolution. Reflection API - introspecting class members at runtime. OOP mini-project integrating multiple classes• Intro to design patterns - factory, strategy, and template	Concepts of inheritance and types, final keyword, polymorphism, Method overriding, multiple classes

3.6 CO6 Attainments

CO6 Syllabus	CO6 concepts included in project
Types of exceptions, hierarchy, try-catch-finally, throw and throws Custom exception classes. File handling: byte stream and character stream. Reading/writing files using FileInputStream, FileOutputStream, FileReader, FileWriter, and buffered streams Serialization and Deserialization. Generics - parameterized classes and methods. Java Collections Framework - List, Set, Map, Queue and their implementations (ArrayList, HashSet, HashMap, PriorityQueue, etc.). Functional Programming in Java Lambda expressions, Stream API	Arraylist , Hashmap ,list , Functional programming(lambdas function interfaces, Generics - parameterized classes and methods

Capstone mini-project integrating file I/O, collections, and exception handling	

3.1.1 Scenario's for CO1 implementation.

CO1 Concepts included	Scenario
Introduction to Java programming model (syntax, variables, data types, type casting, operators) Input/output operations (Scanner, BufferedReader) Control flow: if else ladder ;switch Iterative constructors: for loop; while loop	Conditionals(if/else): ScanResult.calculateRisk(..) uses an if- else ,if else ladder to decide risk level; isSuspiciousURL() uses multiple if checks to flag suspicious URLs. For(Maps.Entry<string,String>entry: SCAM_PATTERNS.entrySet()) iterates pattern categories; while(matcher.find()) iterates regex matches

3.1.2 Scenario's for CO2 implementation.

CO2 concepts included	Scenario
Merge sort	Apply Merge Sort to organize scam matches alphabetically or by length.

3.1.3 Scenario's for CO3 implementation.

CO3 concepts included	Scenario
String methods, Recursion, String Builder, Pattern Matching, Text validation	Normalization with toLowerCase() • The URL is converted to lowercase • This ensures detection is case-insensitive. Example: split ("\\.") used to count domain parts in suspicious URLs. Example: matches () used to detect patterns like IP addresses or repeated hyphens. Example: trim() used when extracting matched scam text. Example: length() used to calculate the

	size of detected scam indicator StringBuilder used in displayResults() to build the scam report text efficiently line by line. Recursion used in mergeSort() and mergeSortByLength() to divide and sort scam matches by category or text length. Text validation (message(isEmpty()) ensures that the user's input message is not empty or invalid before running scam detection.

3.1.4 Scenario's for CO4 implementation.

CO4 concepts included	Scenario
Constructors, return type, methods, parameters, Access specifiers and encapsulation Static data and methods	The constructor is the special method that initializes the GUI window when an object of ScamDetectorGUI is created. It sets up the frame properties and calls the method to build the interface. private static final Map SCAM_PATTERNS = new LinkedHashMap<>(); Scam patterns are shared across all objects. Declaring them static avoids duplication and ensures consistency. private void createUI() method with no return type. private JButton createButton(String text, Color color) method with parameters and return type (JButton). Methods modularize tasks (UI creation, scanning, sorting). Parameters allow customization (button text/color). Return types give back results (like a new button).

3.1.5 Scenario's for CO5 implementation.

CO5 concepts included	Scenario
Concepts of inheritance and types, final keyword, polymorphism, Method overriding, multiple classes	public class ScamDetectorGUI extends JFrame. ScamDetectorGUI inherits from JFrame so you can build your scam detector as a ready-made window Declared final so the reference to the map cannot be changed, protecting the integrity of your detection rules.

3.1.6 Scenario's for CO6 implementation.

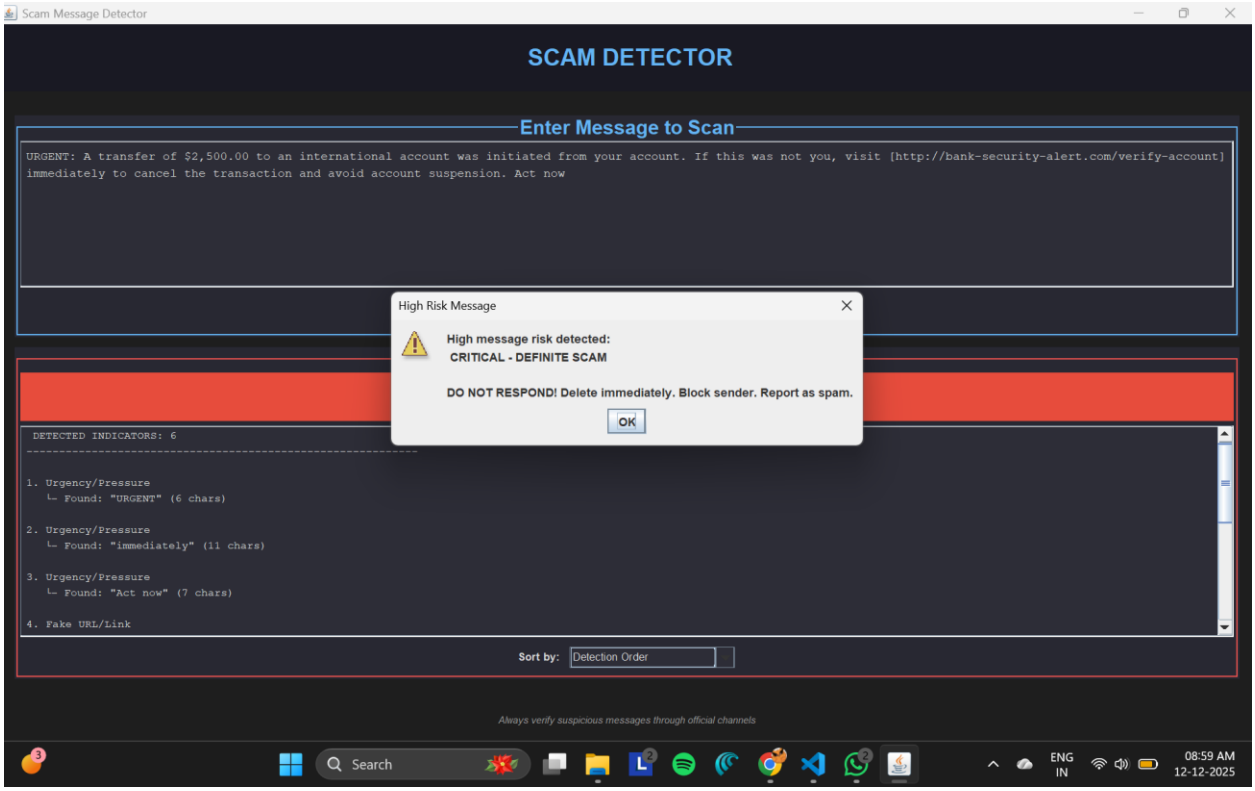
CO6 concepts included	Scenario
Arraylist , Hashmap ,list , Functional programming(lambdas function interfaces, Generics - parameterized classes and methods	ArrayList as a temporary storage while merging sorted sublists. It allows dynamic resizing and easy addition of elements during sorting. Using Java collections (ArrayList, HashMap, List) and functional programming features (lambdas, generics) to build a flexible, type-safe, and efficient scam detection system with clean event handling and reusable sorting logic."

CHAPTER -4 SCREEN SHOTS

4.1 Screen Shots

Overview: The Scan Message function firstly scans the given message and then displays the level of risk i.e. Critical, High, Low, Medium, or No Risk.

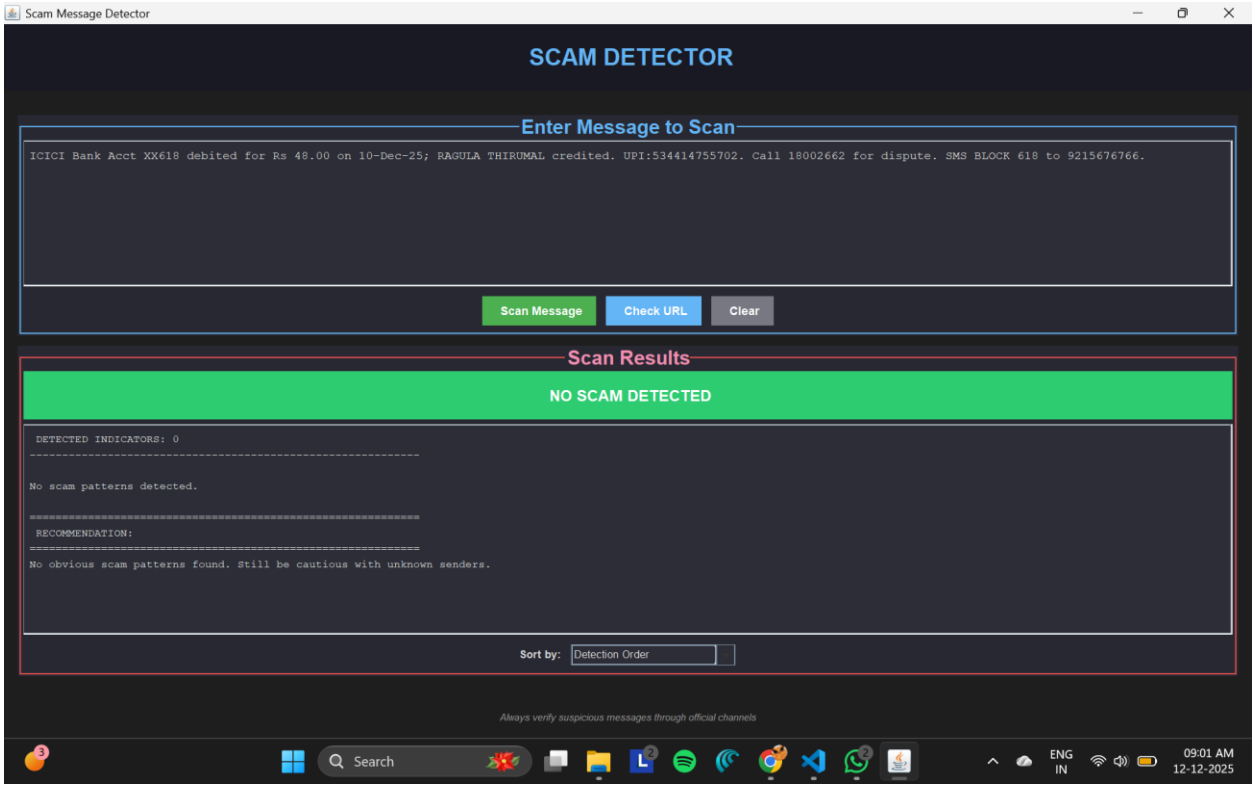
It also scans the URLs and identifies them as High, Medium, Low, or No Scam.



Test Case 1:

The Scan Message function scanned the given message.

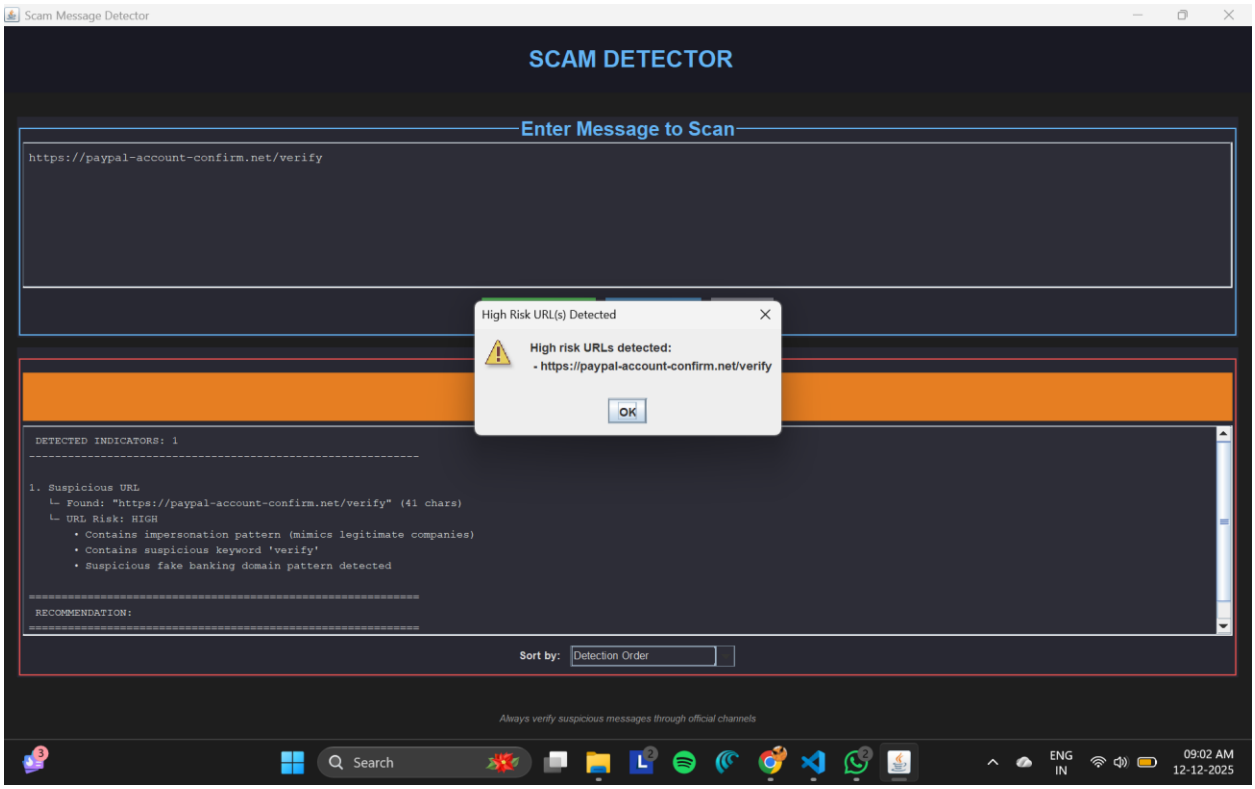
Test case1 resulted in Critical Risk indicating DEFINITE SCAM.



Test Case 2:

The Scan Message function scanned the given message.

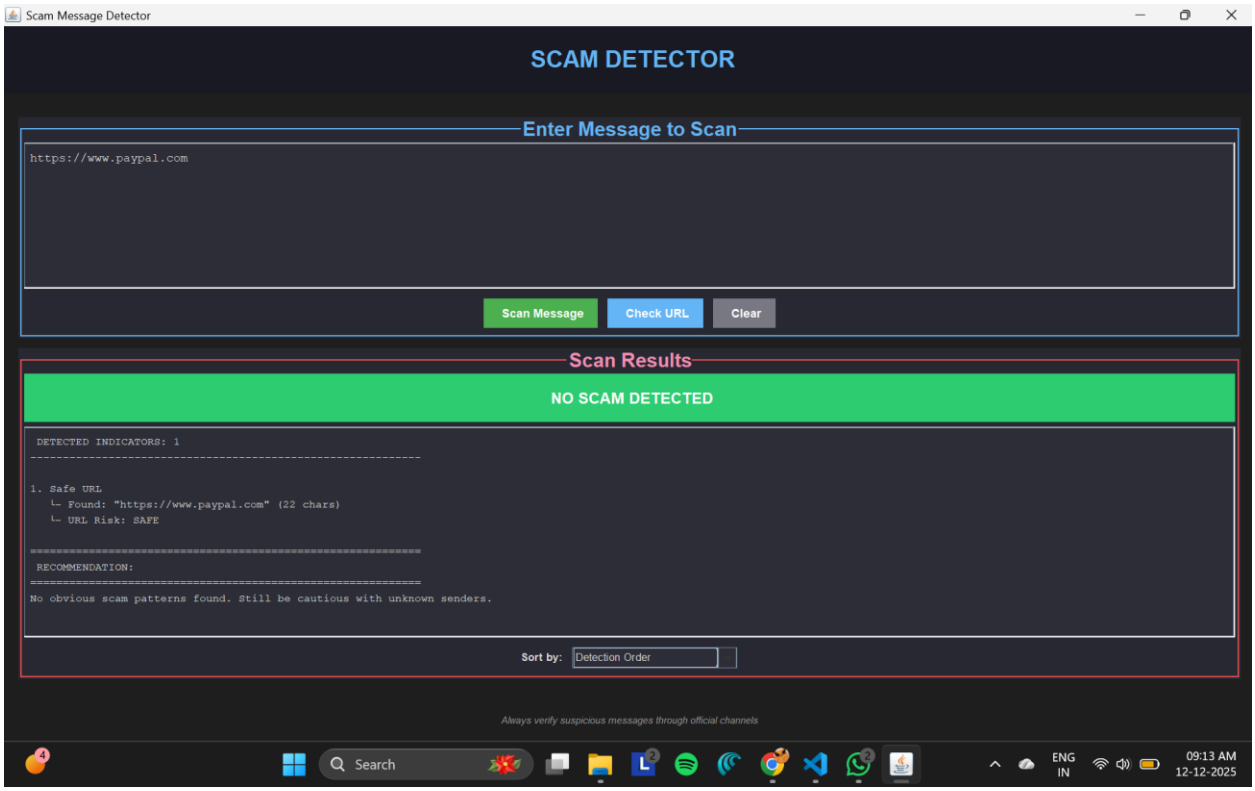
The Test Case 2 is identified to have No risk i.e. NO SCAM DETECTED.



Test Case 3:

The Scan URL function scans the given URL.

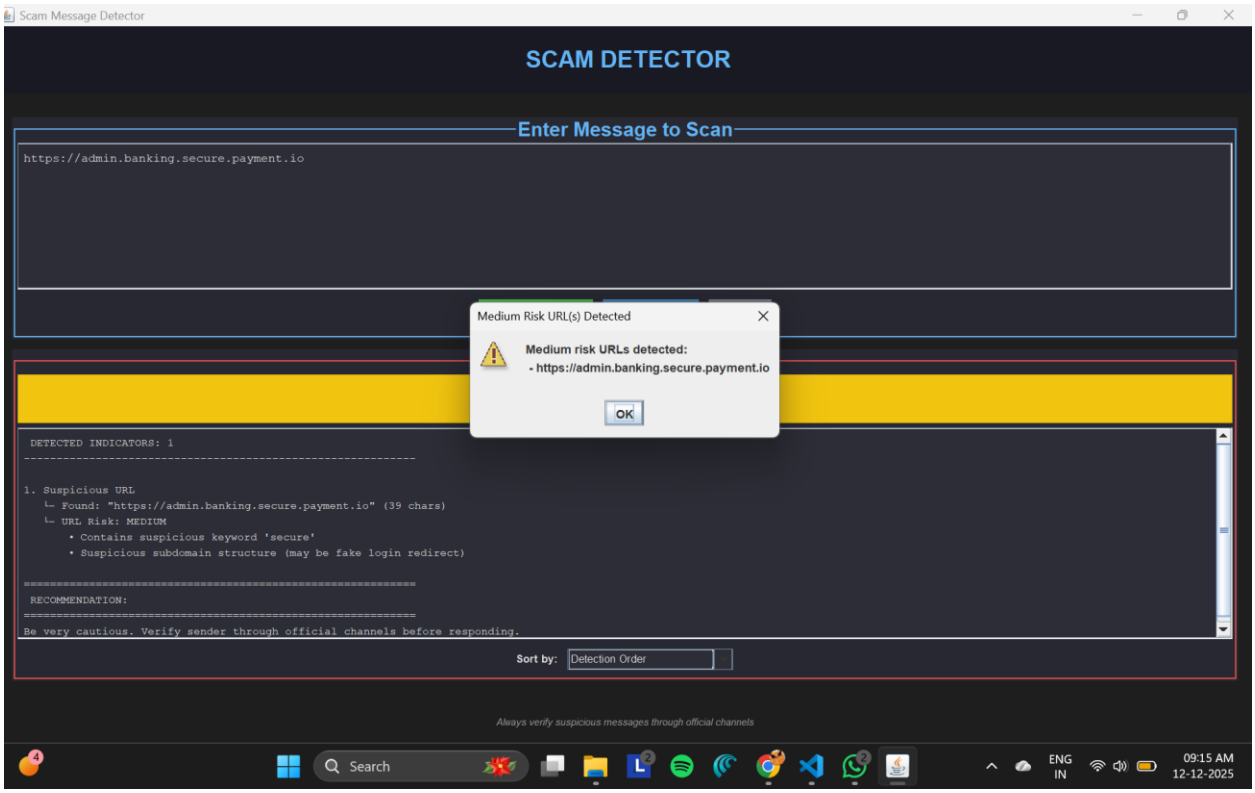
Test Case 3 resulted in High Risk i.e. DEFINTE SCAM



Test Case 4:

The Scan URL function scans the given URL.

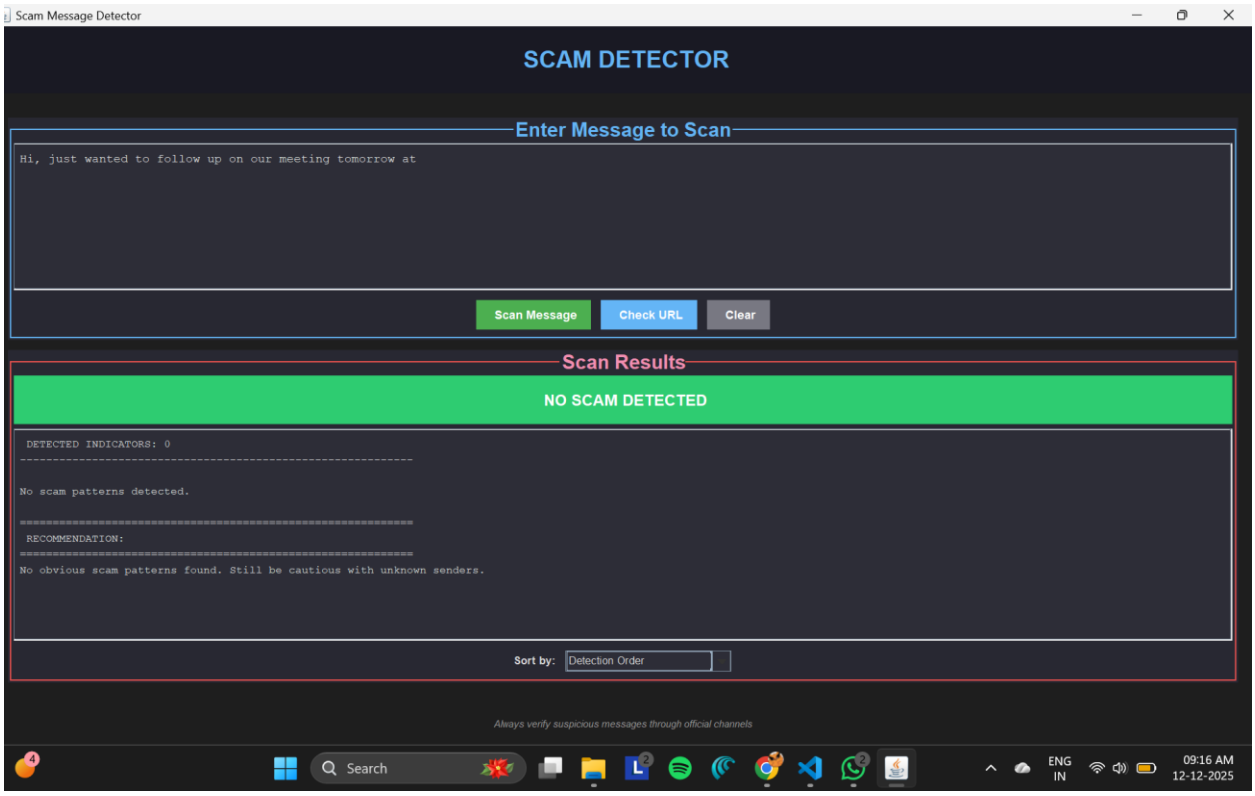
Test Case 4 is identified as no risk, i.e. NO SCAM DETECTED.



Test Case 5:

The Scan URL function scans the given URL.

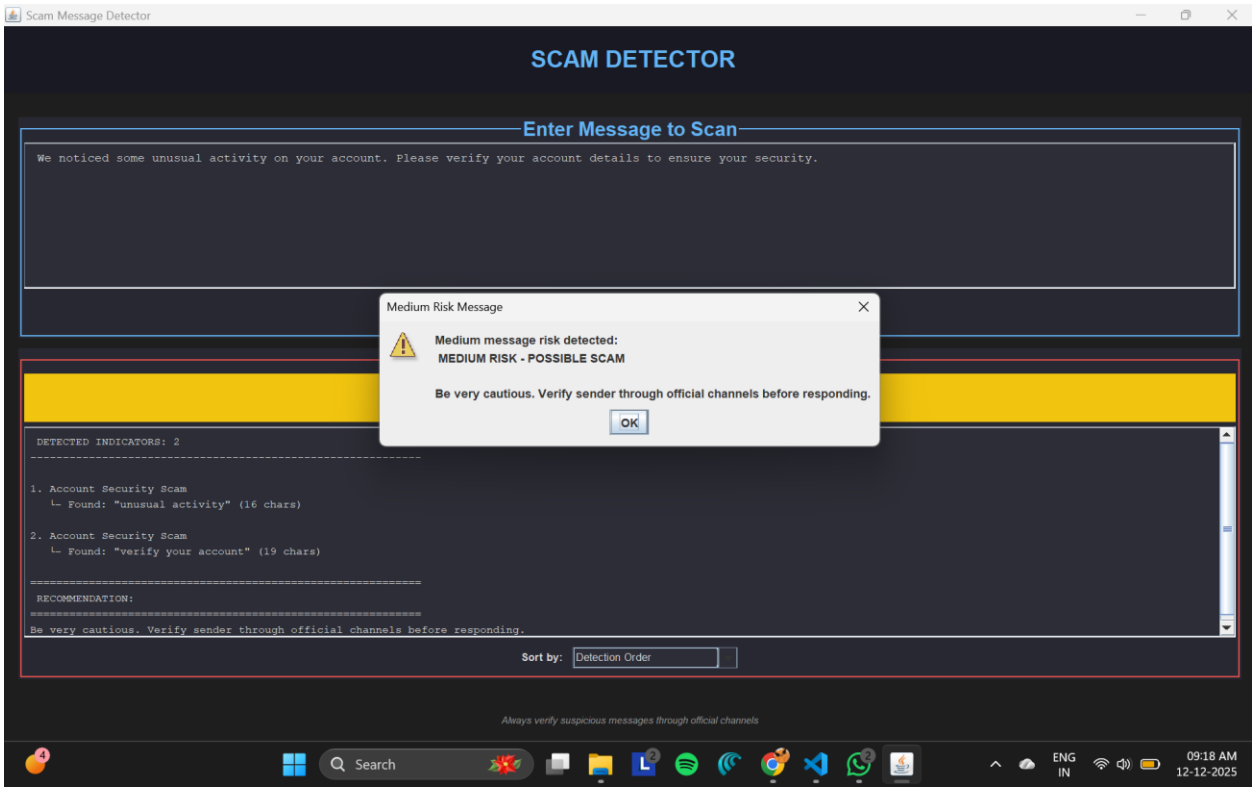
Test Case 5 was found to be Medium Risk, i.e. POTENTIAL SCAM.



Test Case 6:

The Scan Message function scans the given message.

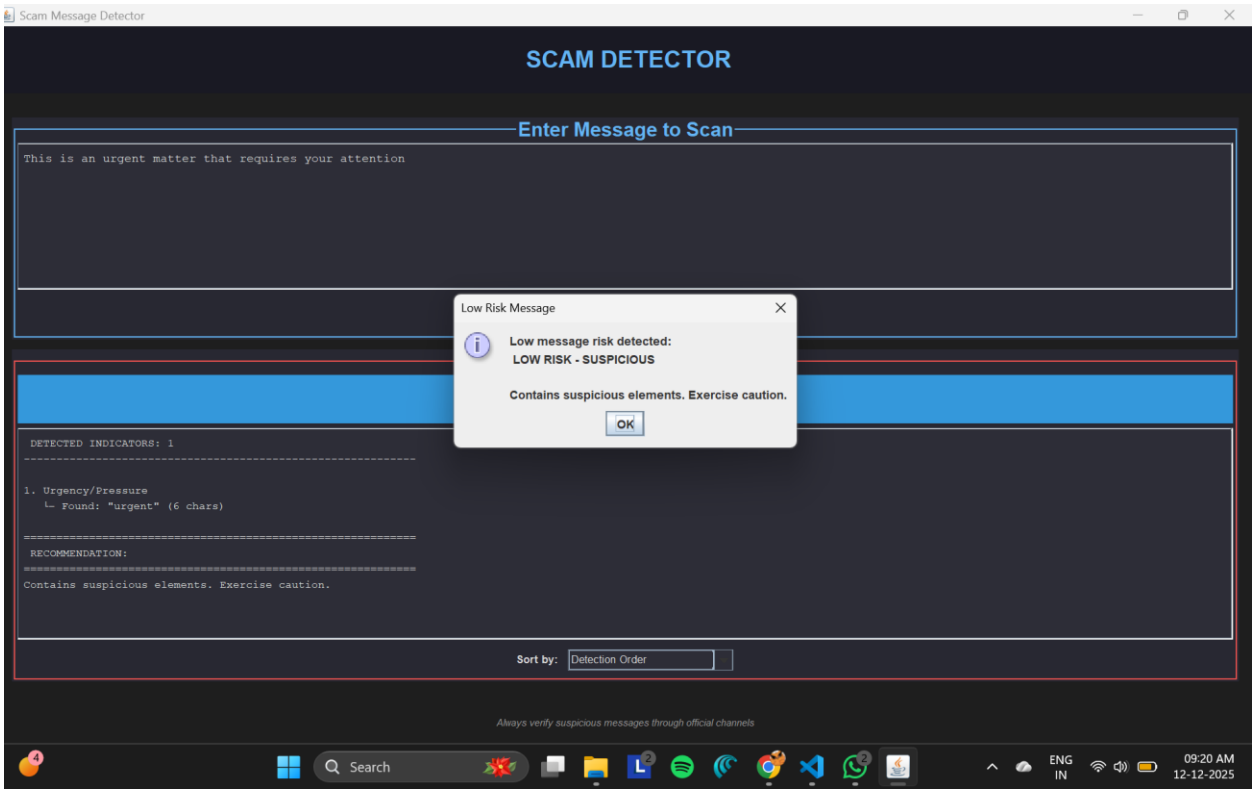
Test Case 6 resulted in No Risk, i.e. NO SCAM DETECTED.



Test Case 7:

The Scan Message function scans the given message.

Test Case 7 is identified as Medium Risk, i.e. Possible Scam.



Test Case 8:

The Scan Message function scans the given message.

Test Case 8 results in Low Risk, i.e. SUSPICIOUS or POTENTIAL SCAM.

CHAPTER -5 TESTING

5.1 Test Cases and Results

S No.	Test Cases	Test Case Description	Prerequisites	Steps	Input Data	Expected Result	Actual Result	Remarks/Status
	TC - 1	Test for multiple scam patterns triggering highest risk level	1. Application is running 2. Message contains urgency, prize, threat, and phishing patterns 3. At least 5 scam patterns present	1. Enter complex scam message 2. Click "Scan Message"	URGENT: A transfer of $2,500.00 to an international account was initiated from your account. If this was not you, visit [http://bank-security-alert.com/verify-account] immediately to cancel the transaction and avoid account suspension. Act now	CRITICAL RISK	**CRITICAL RISK**	**Pass** - System correctly identified definite scam
	TC - 2	Test message with no scam patterns	1. Application is running 2. Message contains no scam pattern keywords 3. All text is legitimate banking information	1. Enter normal message 2. Click "Scan Message"	ICICI Bank Acct XX618 debited for Rs 48.00 on 10-Dec-25; RAGULA THIRUMAL credited. UPI:534414755702. Call 18002662 for dispute. SMS BLOCK 618 to 9215676766.	NO SCAM DETECTED	**NO SCAM DETECTED**	**Pass** - Correctly identified safe message
	TC - 3	Test for high-risk URL patterns	1. Application is running 2. Message contains URL shortener 3. URL is suspicious	1. Enter suspicious URL 2. Click "Check URL"	https://paypal-account-confirm.net/verify	HIGH RISK	**HIGH RISK**	**Pass** - Correctly flagged URL shortener as high risk
	TC - 4	Test legitimate URL	1. Application is running 2. Message contains legitimate URL 3. URL has no suspicious characteristics	1. Enter normal URL 2. Click "Check URL"	https://www.paypal.com	NO SCAM DETECTED	**NO SCAM DETECTED**	**Pass** - Correctly identified safe URL
	TC - 5	Test URL with suspicious characteristics	1. Application is running 2. URL has suspicious	1. Enter URL with multiple subdomains	https://admin.banking.secure.payment.io	MEDIUM RISK	**MEDIUM RISK**	**Pass** - Correctly identified potential scam URL

			characteristics but not critical 3. URL contains suspicious keywords or structure	2. Click "Check URL"				
	TC - 6	Test message containing URL but no scam patterns	1. Application is running 2. Message contains URL but no scam patterns 3. Text is normal communication	1. Enter message with safe URL 2. Click "Scan Message"	Hi, just wanted to follow up on our meeting tomorrow	NO SCAM DETECTED	**NO SCAM DETECTED**	**Pass** - Correctly distinguished between URL and scam patterns
	TC - 7	Test for multiple scam indicators	1. Application is running 2. Message contains 2-3 scam patterns 3. Not enough for critical/high risk	1. Enter message with scam patterns 2. Click "Scan Message"	We noticed some unusual activity on your account. Please verify your account details to ensure your security.	MEDIUM RISK	**MEDIUM RISK**	**Pass** - Correctly identified possible scam
	TC - 8	Test for single scam indicator	1. Application is running 2. Message contains exactly 1 scam pattern 3. Minimal suspicious content	1. Enter message with one scam pattern 2. Click "Scan Message"	This is an urgent matter that requires your attention	LOW RISK	**LOW RISK**	**Pass** - Correctly flagged as suspicious

CHAPTER -6 FUTURE ENHANCEMENTS

6.1 Planned Features

6.1.1 Real-Time Email Detection (High Priority Feature)

Real-time email detection scans messages instantly upon arrival, performing pre-delivery analysis before they reach the inbox. The system evaluates sender authenticity, content anomalies, embedded links, and urgency indicators typical of social engineering attacks. When threats are detected, it generates instant warnings with context and recommendations. This rule-based approach uses pattern matching and signature detection to identify known scam characteristics, protecting users without relying on them to recognize red flags. The system maintains a regularly updated database of threat signatures to detect emerging scam patterns.

6.1.2 Multi-Model Scam Detection (Medium Priority Feature)

Multi-modal scam detection analyzes multiple data types simultaneously to identify fraudulent content comprehensively. This component processes images, documents, and various file formats, employing image analysis algorithms to detect fake logos, forged screenshots, and malicious QR codes. The system examines visual elements for brand inconsistencies and manipulated images while tracking sender behavior patterns including email frequency, domain reputation, and communication style. By combining visual and behavioral signals through rule-based analysis, it reduces false positives while catching sophisticated scams that single-channel methods miss.

6.1.3 Real-Time Website Analysis (Low Priority Feature)

Real-time website analysis operates through a browser extension that monitors sites before user interaction. The feature uses visual similarity comparison to check visited websites against a database of legitimate sites, identifying spoofed pages that mimic trusted brands. It flags suspicious data requests and behaviors associated with credential harvesting based on predefined security rules. When threats are detected, the system provides instant warnings with threat levels ranging from low risk to critical, including risk explanations and recommended actions. This enables quick, informed decisions and protects users during vulnerable moments like

entering sensitive information.

CHAPTER -7 CONCLUSION

7.1 Summary of the Project

The Scam Detection System has successfully established a functional foundation for protecting users against online scams and fraudulent activities. The current implementation features two core modules: text-based scam detection that identifies suspicious communication patterns and urgency tactics, and URL analysis that detects malicious links and phishing attempts through domain verification and pattern recognition.

Developed entirely in Java, the system employs rule-based algorithms and pattern matching techniques to deliver reliable threat detection. The modular design ensures easy maintenance and allows for future enhancements without requiring extensive restructuring. By maintaining updated threat databases, the system effectively identifies both common and emerging scam patterns.

The project demonstrates that practical and effective scam detection can be achieved through straightforward technical approaches. With planned additions such as real-time email scanning, image analysis, and browser extension capabilities, the system is positioned to evolve into a comprehensive security solution. The current success of text and URL detection validates the project's approach and provides a strong foundation for future development, ultimately contributing to safer online experiences for users.

CHAPTER -8 REFERENCES

Books

O'Reilly Media. Marc Loy, Patrick Niemeyer, Daniel Leuck (2023) *Learning Java: An Introduction to Real- World Programming with Java* (6th ed.)

Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw- Hill Education.

Herbert Schildt (2024) *JAVA THE COMPLETE REFERENCE (13th edition.)* McGraw- Hill Education.

Tutorials and Learning Resources

W3Schools. (2025). *JavaScript Tutorials.* Retrieved from https://www.w3schools.com

GeeksforGeeks.(2025) .*UML Class Diagram.* Retrieved from

https://www.geeksforgeeks.org

Documentation and Research References

Crime Pattern Detection Using Data Mining (2006). Retrieved from

https://ieeexplore.ieee.org/document/4053200

Lightweight deep learning model for crime pattern recognition based on transformer with simulated annealing sparsity and CNN (2025). Retrieved from

https://www.nature.com/articles/s41598-025-07260-7

CHAPTER -9 APPENDICES

9.1 Installation / Setup Instructions

Step 1: Install java

1) Install the java development kit (JDK 8 or above) on your system

2) After the installation you can check it by using the following command in the terminal.

```
nginx


java -version
```

Step 2: Open a java ide

1) Open any Java editor such as NetBeans, IntelliJ IDEA, Eclipse, or VS Code.
2) Create a new Java project and add the project source files to it.

Step 3: Add input text file

1) Prepare a text file that you want to scan for suspect pattern
   Place this file inside the project folder it will be easy to access

Step 4: Run the program

1) Right-click on the main Java file and select Run.
2) When the program starts, it will ask for the path of the text file to scan
3) Enter the correct path and press Enter.

Step 5: View the output

The program will scan the text line-by-line and display:

- The suspect keywords found
- The line numbers where they appear
- The number of matches detected

9.2 USER MANUAL

STARTING THE APPLICATION

Open the project in your java IDE and run the main file

The application will open the in the main terminal

Providing input

Prepare a plain text file that contains the data to be analyzed

When prompted, enter the file path so the system can read the content

HOW THE SYSTEM WORK'S

The application Read the text file line by line Compare's each line with the predefined list of suspect keywords, detect and records any matches found

RESULT

After processing, the system displays Which keyword is found on which line it was found Total number of matches, a simple summary report.

Ending Session

Once the results are displayed, the user can close the program or run it again with a different text file.

Geo Tag photos with guide


GPS Map Camera
Hyderabad, Telangana, India 🇮🇳
Gcw3+wm6, Aleap Industrial Area, Gajularamaram,
Hyderabad, Telangana 500043, India
Lat 17.547364° Long 78.404202°
Friday, 12/12/2025 10:23 AM GMT +05:30
Google